

Lecture 21:
Deep Learning
Advancement in Convolutional Neural
Networks

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- **Recall:** Innovations in Convolutional Neural Networks
- More advancement
 - From classification to localization

Advanced Deep Learning Architectures

- **Early CNNs (AlexNet, VGG):**
 - Hierarchical feature extraction via stacked convolution layers.
 - Great success on ImageNet but limited scalability.
- **Challenges:**
 - Vanishing gradients when depth increases.
 - Overfitting due to large parameters.
 - Fixed receptive field will result to limited context capture.

The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

The ResNet Solution: Separate the Identity from the Learning

- ResNet introduces a *skip connection*:

$$H(x) = F(x) + x$$

- Here:
 - x : the *input*, sent directly to the output (the identity path).
 - $F(x)$: a *residual function*, a small correction.
- So the network no longer needs to learn the entire $H(x)$.
It only needs to learn how to change the input slightly.

Residual Learning: Learn the Change, Not the Whole Function

- Instead of learning $H(x)$ directly, ResNet learns how the output differs from the input.

$$H(x) = F(x) + x$$

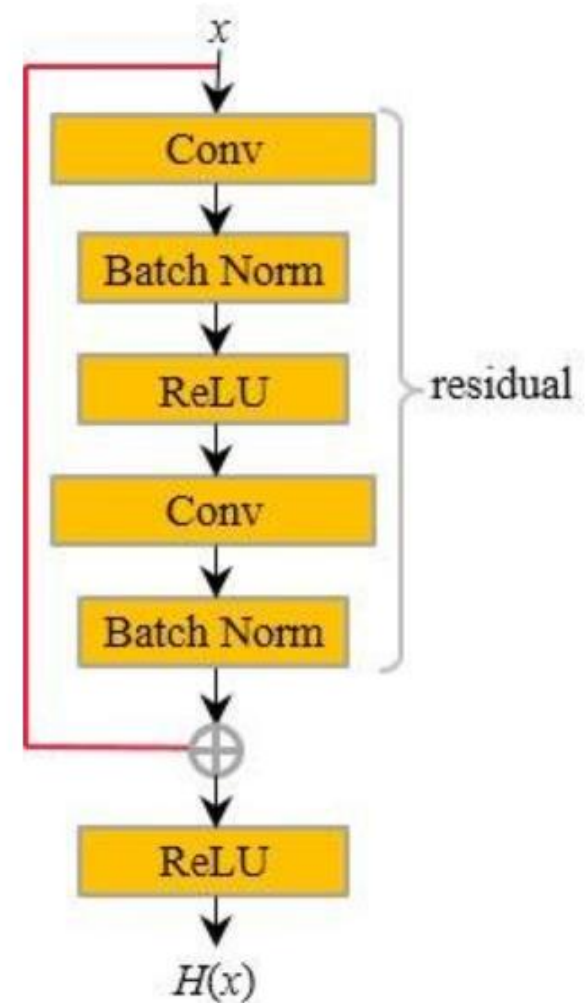
- x : input (identity path)
 - $F(x)$: residual (correction or change)
- If $F(x) = 0$ block acts as identity.
- If $F(x) \neq 0$ block learns meaningful adjustment.

The Building Unit of ResNet

- Structure:
 - Two 3×3 convolutional layers + BatchNorm + ReLU.
 - Skip connection adds input to the output of these layers.

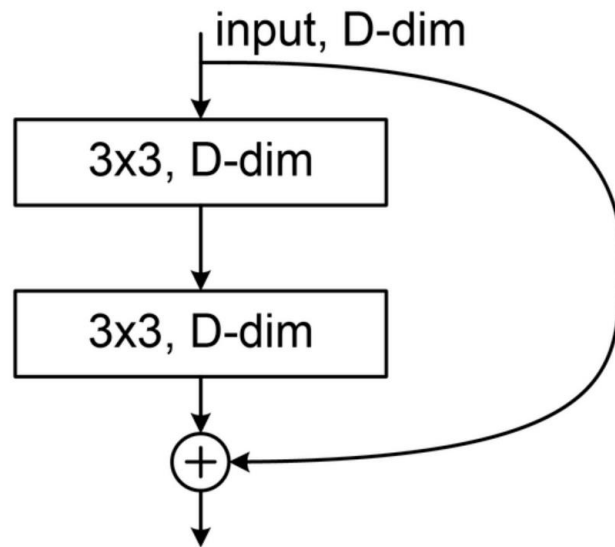
Used in **ResNet-18** and **ResNet-34**.

Single Residual Block

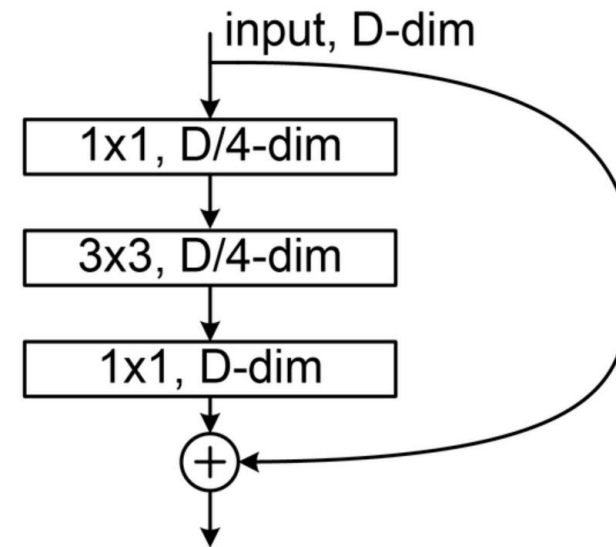


The Bottleneck Block (for Deeper ResNets)

- Deeper networks (ResNet-50, 101, 152) are computationally expensive.
- **Solution:** Add 1×1 (kernel size) convolutions to reduce and then expand channel depth around the 3×3 layer.



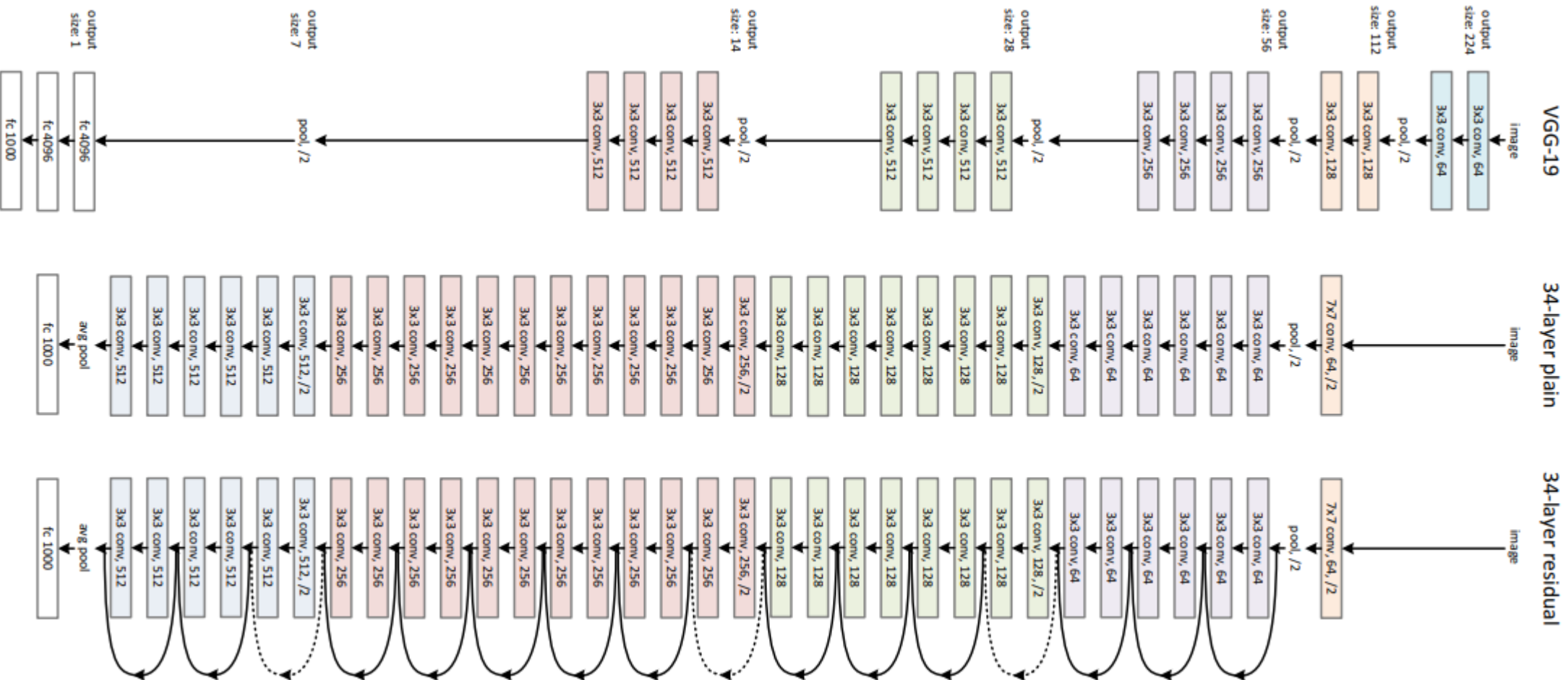
Single Residual Block



Bottleneck Block

$$y = W_3 \sigma(BN(W_2 \sigma(BN(W_1 x)))) + x$$

ResNet-34



The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

2

The Core Idea of Dense Connectivity

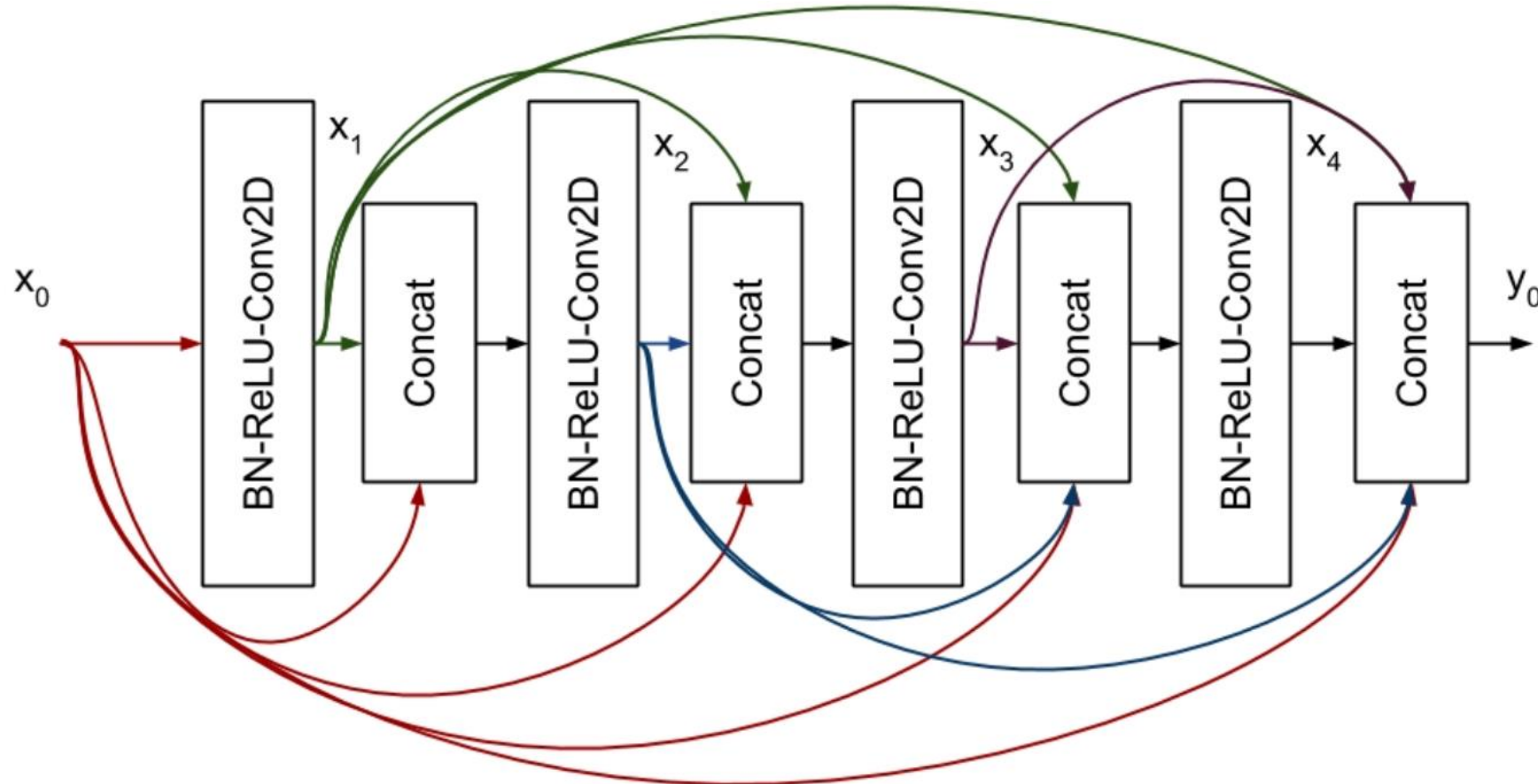
- Connect Every Layer to Every Other Layer
- DenseNet introduces **dense connections**, where each layer receives as input the *concatenation* of all preceding layers' feature maps.

$$x_l = H_l([x_0, x_1, x_2, \dots, x_{l-1}])$$

- $[]$ = concatenation along the channel dimension.
 - H_l = a composite function (BN --- ReLU --- Conv).
 - Every layer **reuses** earlier features directly, no relearning.
- **Key Difference from ResNet:**
 - ResNet: adds features: $x + F(x)$
 - DenseNet: concatenates features: $[x, F(x)]$

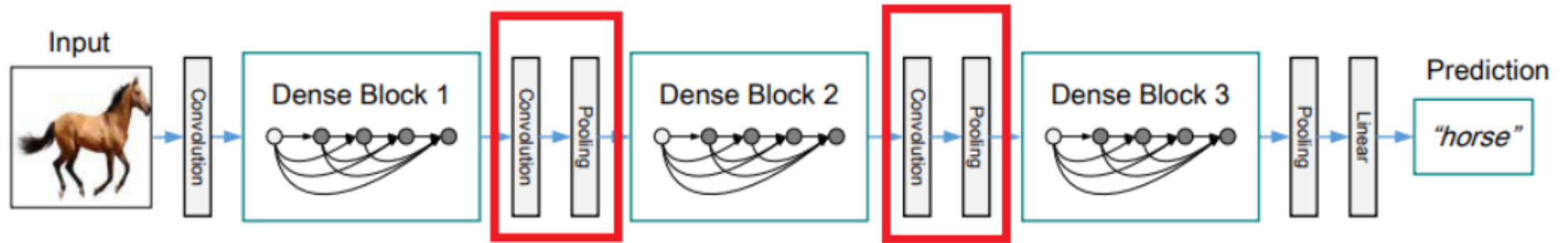
Inside a Dense Block

A **Dense Block** is a group of layers with dense connectivity.



A 4-layer Dense block in DenseNet.

Transition Layers



A deep DenseNet with three dense blocks and two transition layers

The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
3 Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

The Inception Idea

- Multi-Scale Processing Through Parallel Filters
- Inception modules process the same input through **multiple convolutional paths in parallel**, each with different kernel sizes.

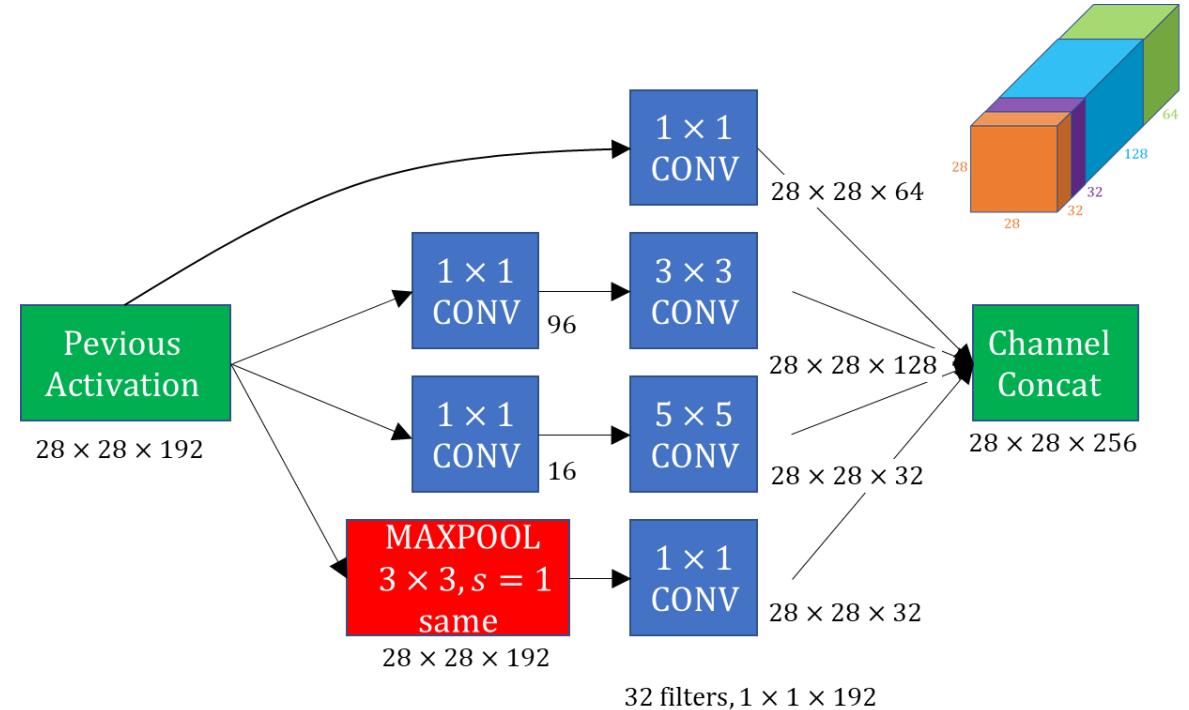
$$\text{Output} = [\text{Conv}_{1 \times 1}(x), \text{Conv}_{3 \times 3}(x), \text{Conv}_{5 \times 5}(x), \text{Pooling}(x)]$$

Concatenate all outputs along the channel dimension.

- Each path captures information at a different receptive field size.
- The network learns *which scale matters* for each feature.

Architecture of an Inception Module

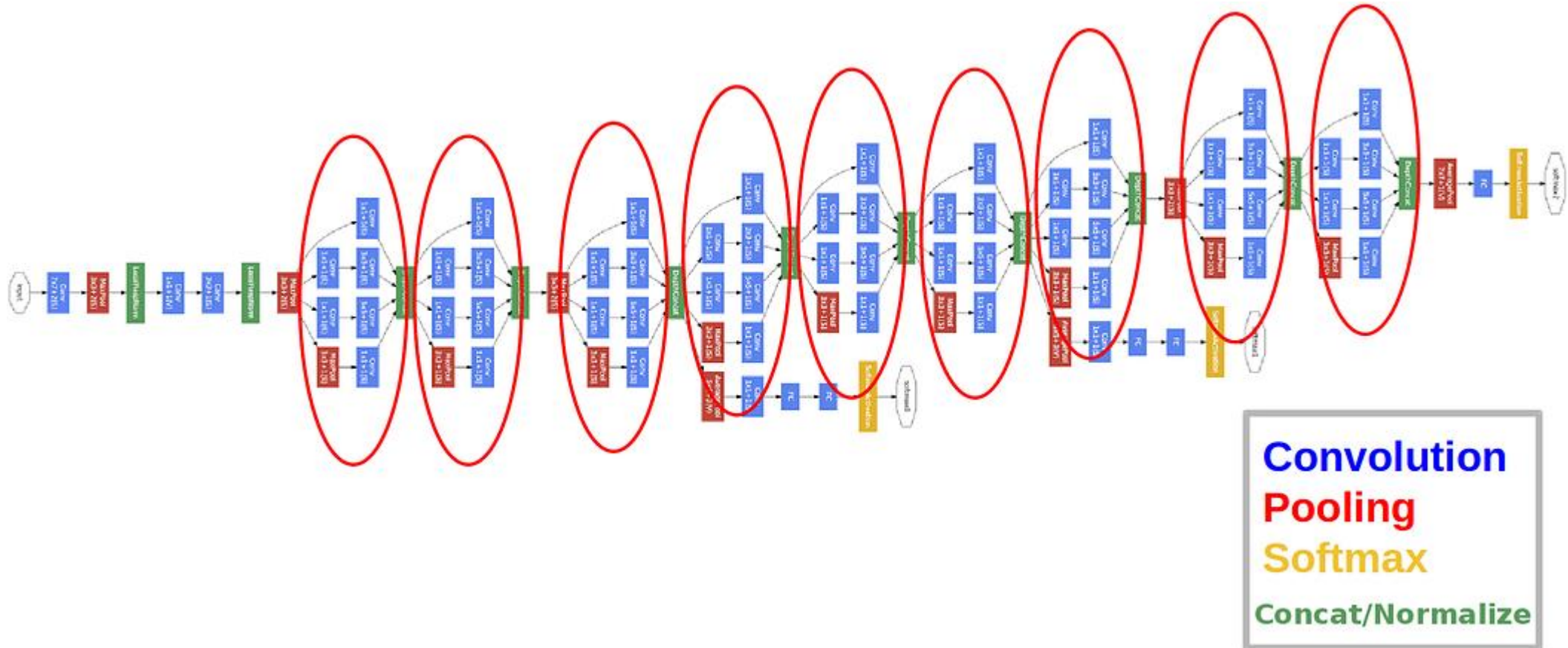
- Branch 1:
1×1 Convolution
- Branch 2:
1×1 Convolution → 3×3 Convolution
- Branch 3:
1×1 Convolution → 5×5 Convolution
- Branch 4:
3×3 Max Pooling → 1×1 Convolution



The outputs from **all four branches** are **concatenated depth-wise** (along the channel axis):

$$\text{Output} = [\text{Branch}_1, \text{Branch}_2, \text{Branch}_3, \text{Branch}_4]$$

GoogleNet



The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
4 Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

Tokens Are the Input Units for Attention

- Split the image into small patches
- Example: 224×224 image: divide into 16×16 pixel patches, you get $(224/16)^2 = 14 \times 14 = 196$ patches.
- Each 16×16 patch becomes one token.
- The Transformer does not see 'the image' or 'the sentence' as a whole. it only sees a sequence of tokens, similar to:

$$[x_1, x_2, x_3, \dots, x_n]$$

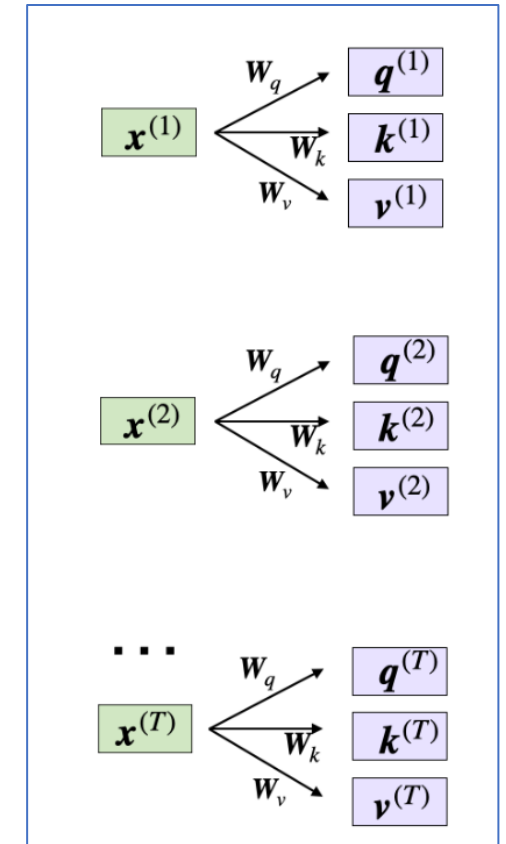
- Each token will:
 - Create its own **query (Q)**: what I am looking for
 - Create its own **key (K)**: what I offer
 - Create its own **value (V)**: what information I contain

Learning Relationships, Not Just Filters

- **Concept: Self-Attention**
- Each pixel (or patch) can attend to every other pixel in the image.
The model learns *which regions are important for interpreting each patch*.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- where
 - Q = query vectors,
 - K = key vectors,
 - V = value vectors,
 - d_k = dimension scaling factor.

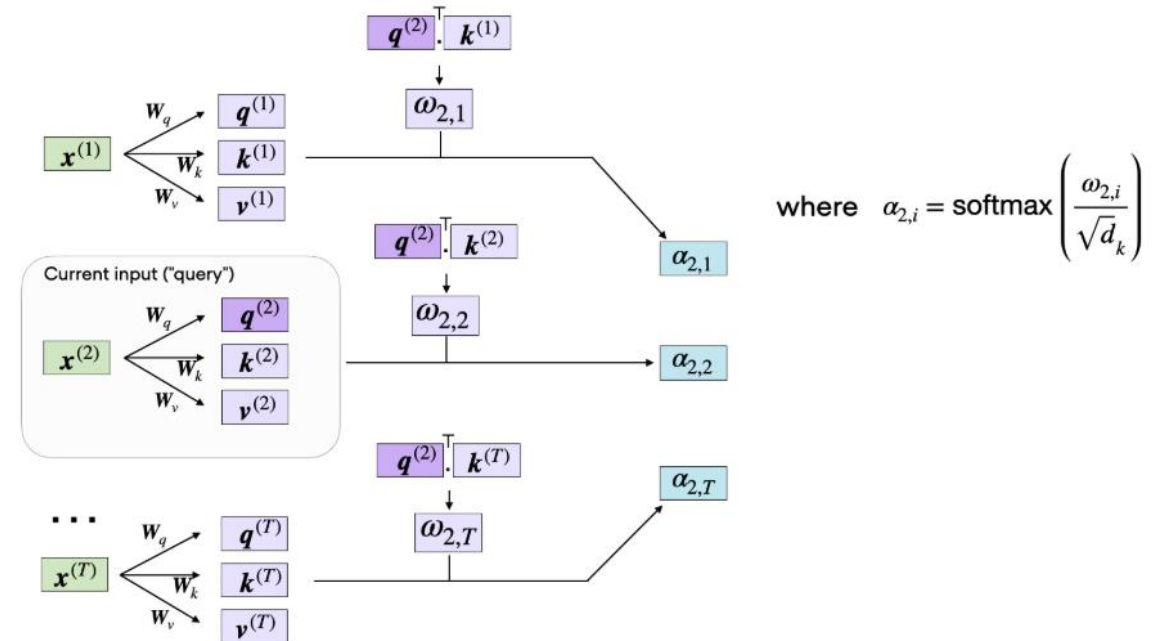


Softmax: Turning Scores into Weights

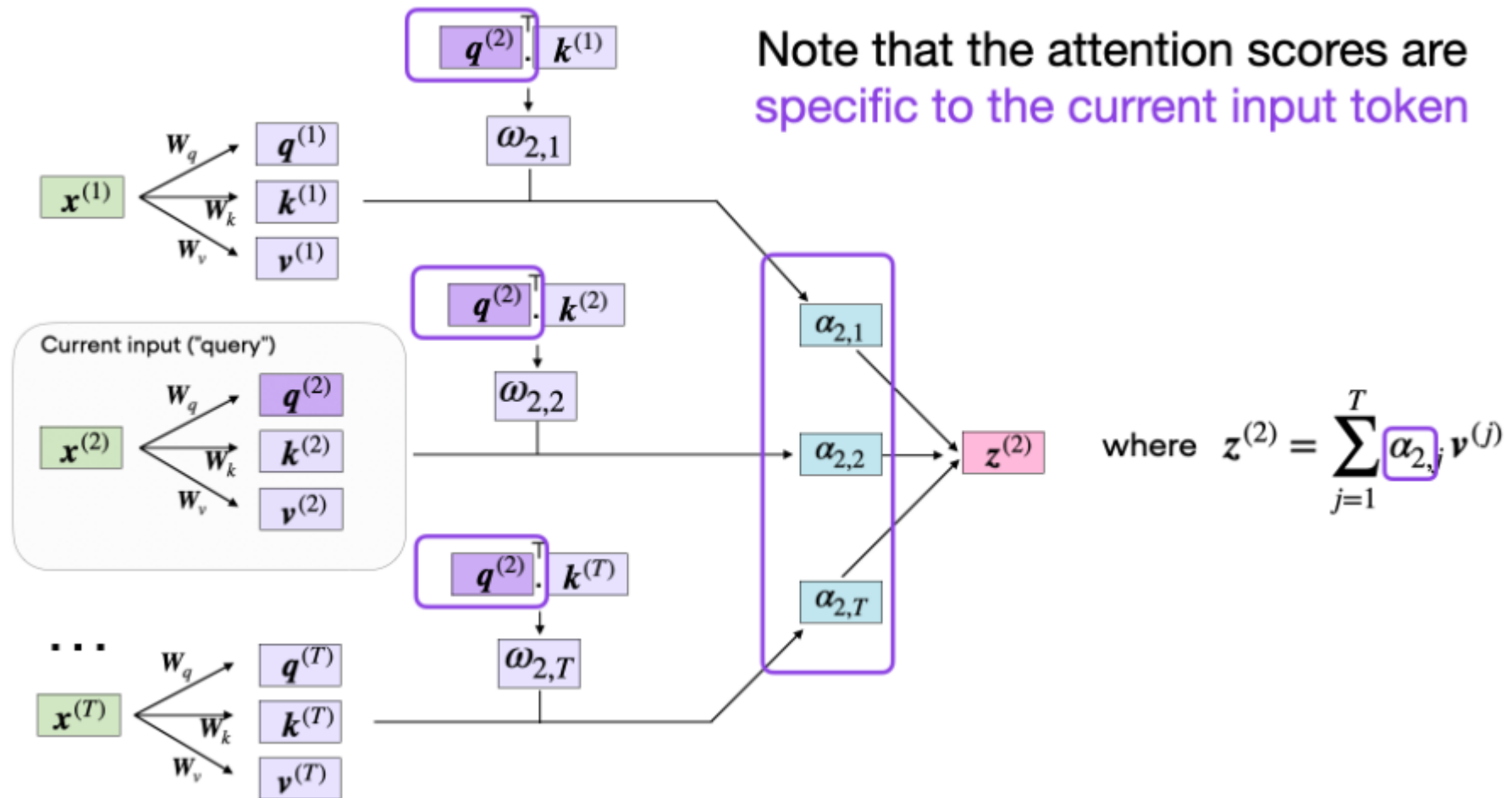
- The softmax function converts each row (all similarities for a given query) into probabilities that sum to 1.
- So for token i , the softmax gives you a set of weights:

$$\text{Attention Weights}_i = \text{softmax}\left(\frac{Q_i K^T}{\sqrt{d_k}}\right)$$

- High weight: token i focuses strongly on token j
- Low weight: token i mostly ignores token j



Attention-weighted version of original query for the second input element

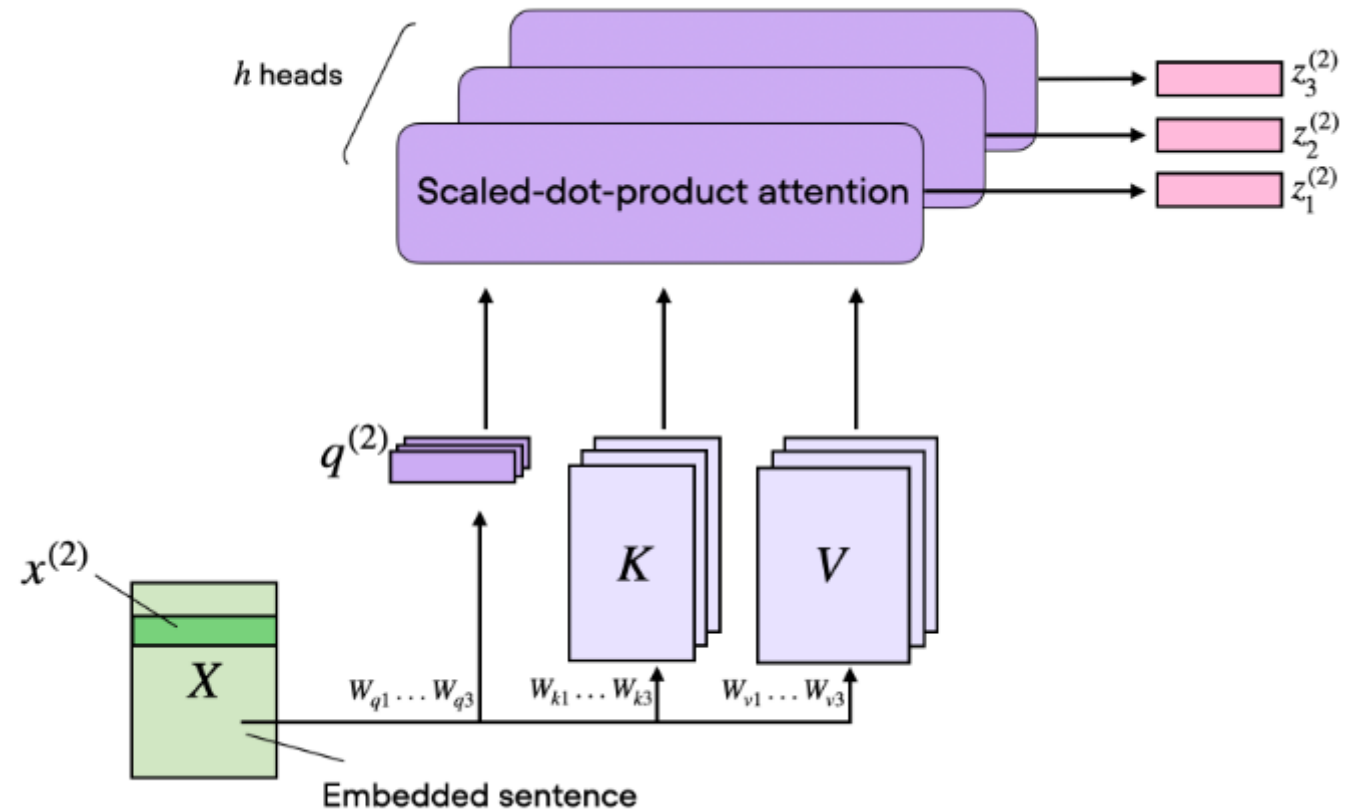


Several Concepts

- Multi-head attention
- Cross-attention

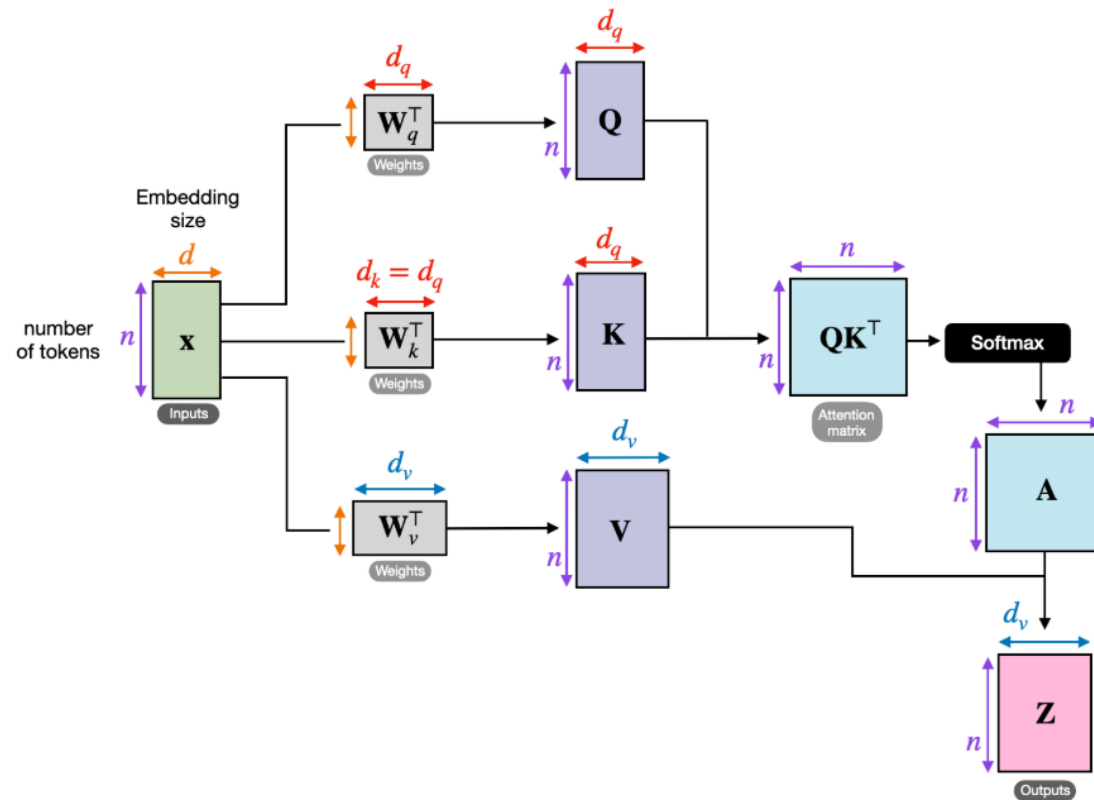
Multi-head Attention

This concept is similar to the use of multiple kernels in convolutional neural networks.



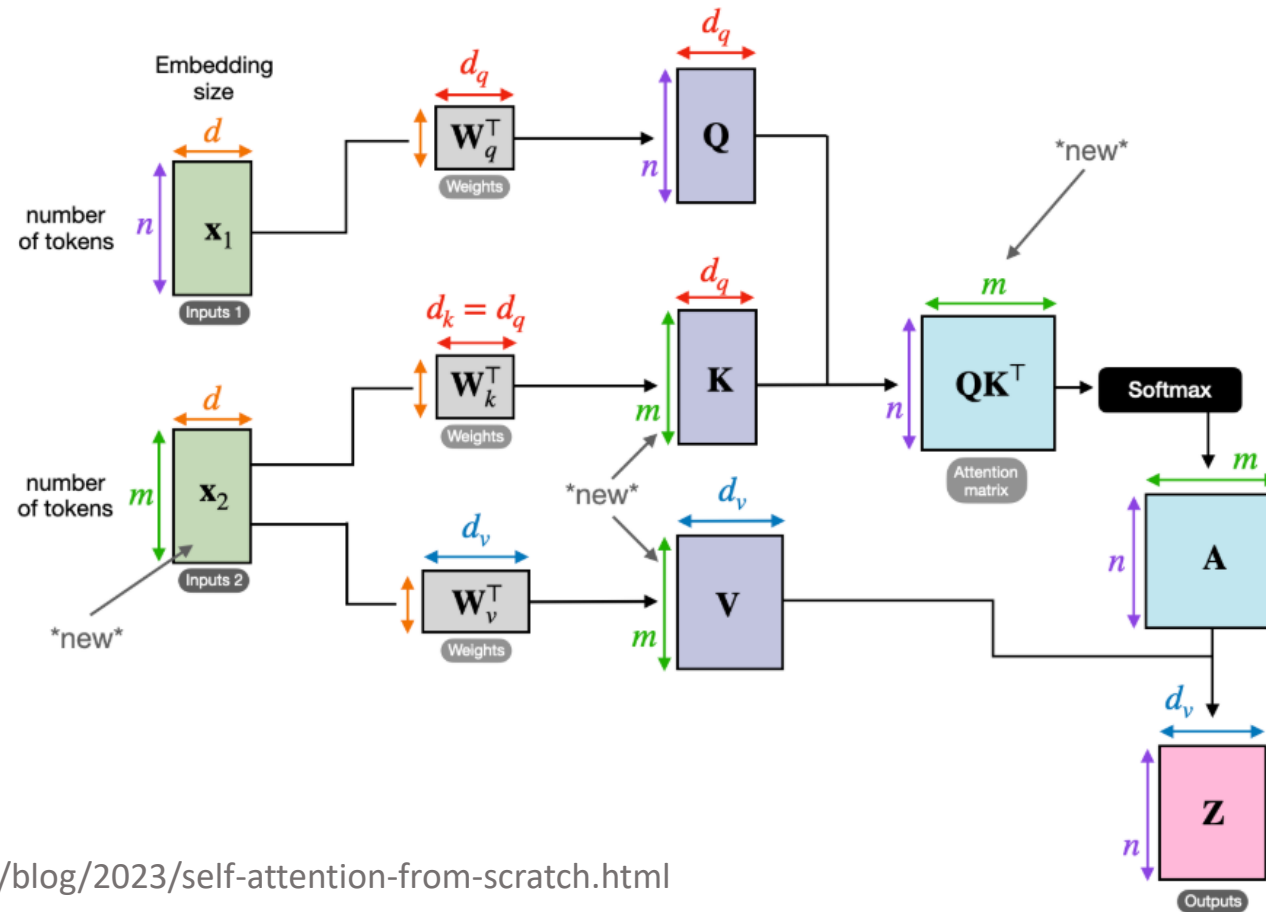
Self-attention

- In self-attention, we work with the same input sequence.



Cross-attention

- In cross-attention, we mix or combine two *different* input sequences



From Classification to Localization

- How CNNs learned to not only see but also localize.
- Two-stage detectors: R-CNN, Fast R-CNN, Faster R-CNN
- One-stage detectors: YOLO, SSD
- Transformer-based: DETR

From Classification to Detection

- Classification gives one label per image (e.g., cat).
- Real-world scenes have multiple objects at different locations.
- We need:

$$\{(b_i, c_i, s_i)\}, i = 1..N$$

Where, Bounding box b_i , class c_i , confidence s_i .

A network must predict both what and where.

Problems:

- Different number of objects per image.
- Objects appear at multiple scales.
- Background clutter.

R-CNN (2014): Regions with CNN

Motivation

- CNNs (e.g. AlexNet) worked for classification: one object.
- How to extend CNNs to find *many objects*?

Mechanism

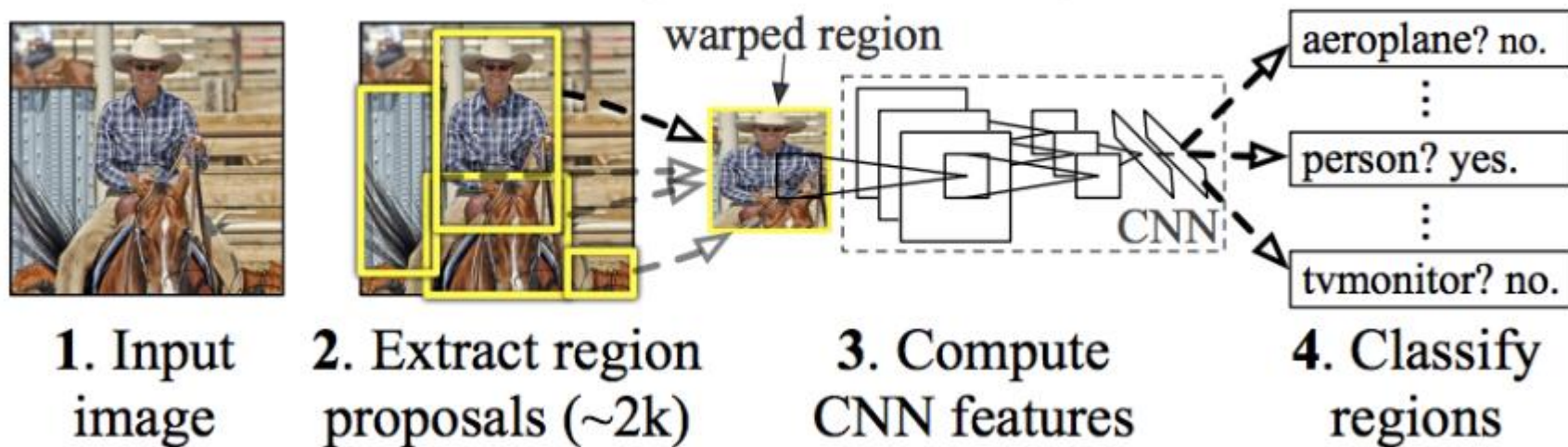
- 1.**Selective Search:** Propose ~2000 candidate regions (ROIs).
- 2.**CNN Feature Extraction:** Run each region through pretrained CNN (e.g., AlexNet).
- 3.**SVM Classifier:** Classify each region.
- 4.**Bounding Box Regression:** Fine-tune box coordinates.

R-CNN

- Before R-CNN, object detection relied on hand-crafted features (HOG, SIFT) and shallow classifiers.
R-CNN was the first to combine deep CNN feature extraction with region-based detection.
- R-CNN is for multiple-object detection, but it does so indirectly by detecting objects one region at a time.
- It does not look at the entire image in one pass (similar to YOLO does later). Instead, it looks at many *proposed regions* and classifies each one as a specific object or background.

R-CNN

R-CNN: *Regions with CNN features*



Fast R-CNN (2015): Shared Features

- **Motivation**

- R-CNN wastes computation: each region runs full CNN.
- Need to reuse features for all regions.

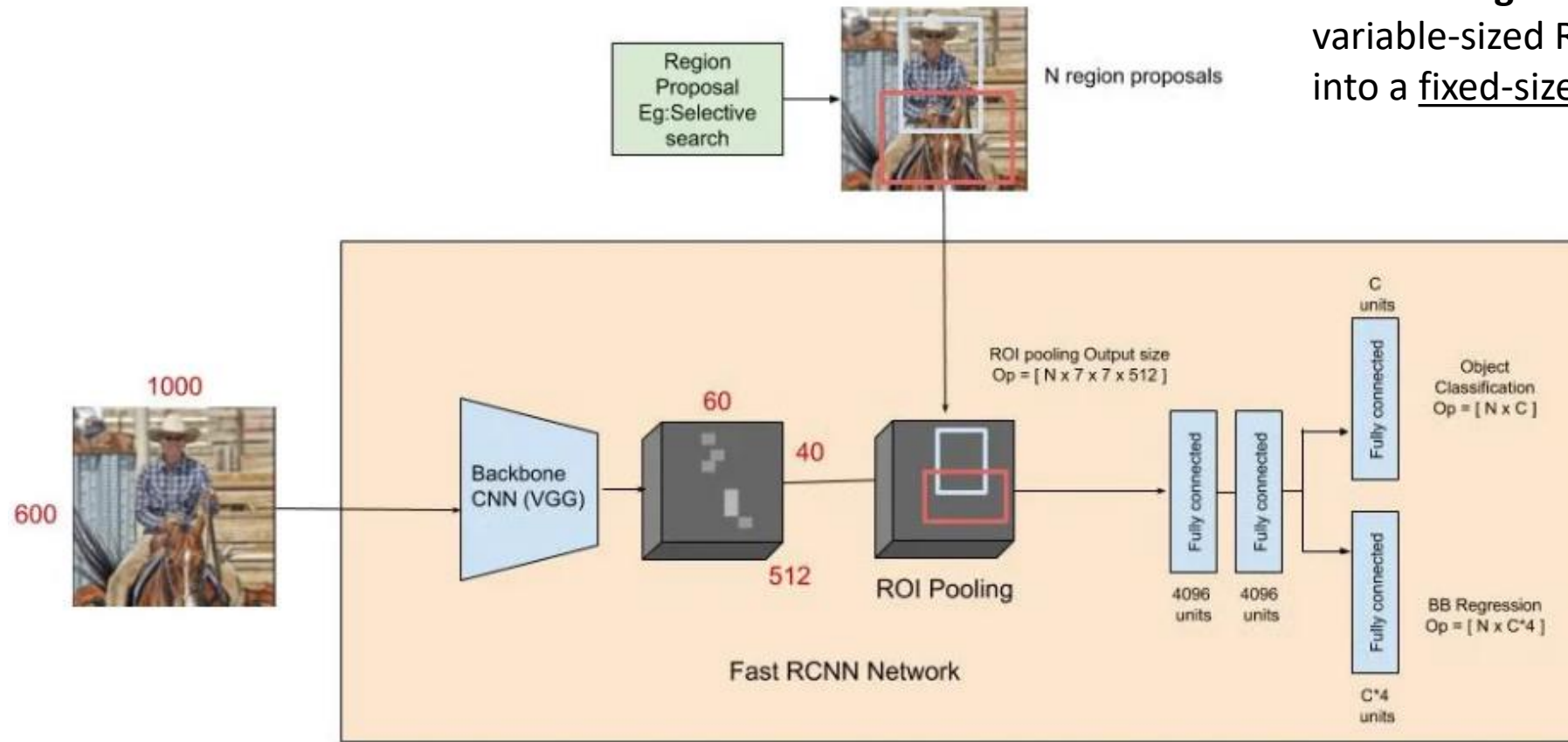
- **Mechanism**

- One CNN over the entire image: shared feature map.
- Each region proposal projected onto this feature map via ROI Pooling.
- Fully connected layers classify and regress box offsets.

Core Mechanism of Fast R-CNN

- **Step 1: Shared Convolutional Feature Map**
 - The entire image is fed once into the CNN backbone (e.g., VGG16).
 - Output: a high-level feature map F of size $H' \times W' \times C$.
 - Each pixel in F corresponds to a receptive field region in the original image.
- **Step 2: Region Proposals**
 - The same region proposals (from Selective Search) are reused: about 2000 boxes.
 - But now, instead of cropping the image, each proposal is mapped to its corresponding region on the feature map F .

Fast R-CNN



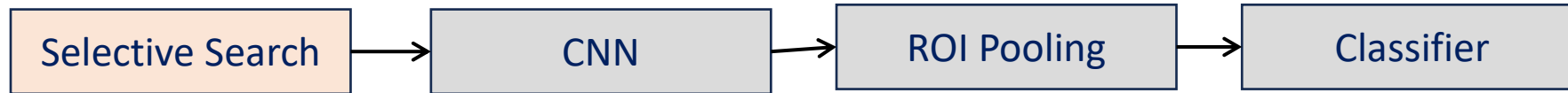
ROI Pooling converts each variable-sized ROI feature patch into a fixed-size feature.

Faster R-CNN: Learning to Propose Regions

- **Why Go Beyond Fast R-CNN?**
- **Fast R-CNN** fixed redundant CNN passes using ROI Pooling, but it still relied on Selective Search for generating ~2000 region proposals.
- **Selective Search Limitations:**
 - Hand-crafted, non-learnable
 - Produces many redundant regions
 - Not adjustable: cannot improve with training data

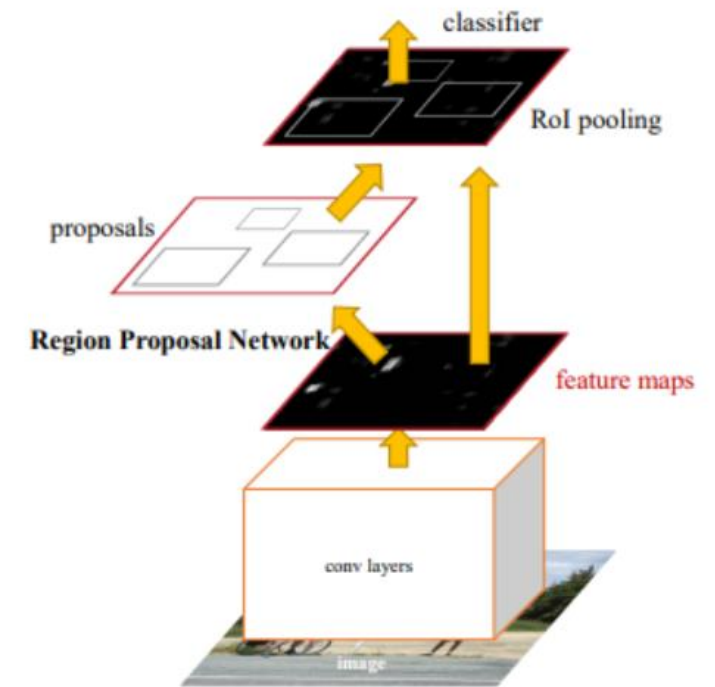
Faster R-CNN

- Introduce a small neural network called a **Region Proposal Network (RPN)** that *learns* to propose candidate object regions directly from CNN feature maps.



Faster R-CNN: Overall Architecture

1. Input Image to Shared CNN Backbone (e.g., VGG16 or ResNet-50)
2. Feature Map to RPN
3. RPN outputs:
 - objectness score (is this region likely to contain an object?)
 - bounding box coordinates
4. The top N proposals (e.g., 300) are sent to ROI Pooling
5. ROI features to classification + refined bounding box regression head



Faster R-CNN architecture

From Two-Stage to One-Stage Detectors

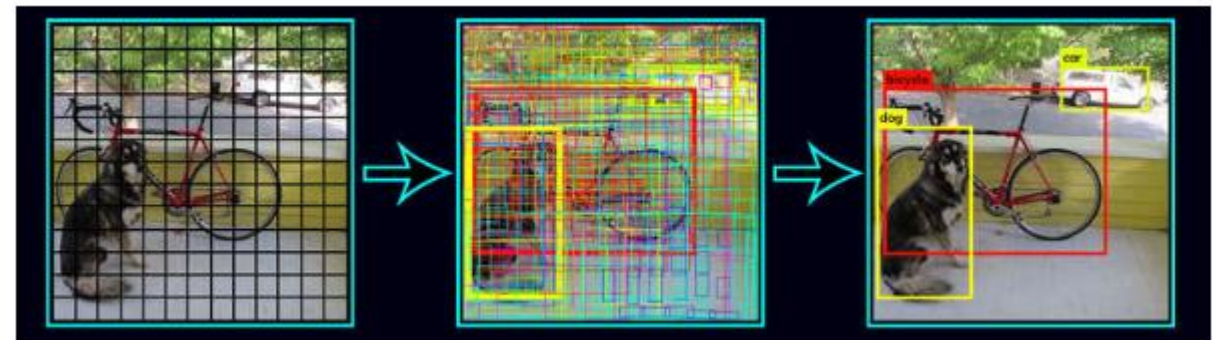
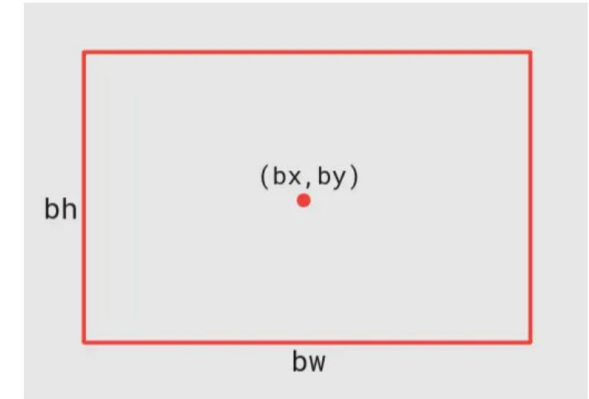
- Two-stage detectors (Faster R-CNN) are accurate but slow.
 - Need region proposals and ROI Pooling and classifier.
- Many real applications (autonomous vehicles, robotics, manufacturing inspection) need *fast inference*.
- **Objective:**
Design a **single-stage** detector that predicts *bounding boxes + classes directly from one CNN pass*.

YOLO: You Only Look Once (2016)

Turn detection into a single regression problem from (image pixels) to (bounding boxes and class probabilities).

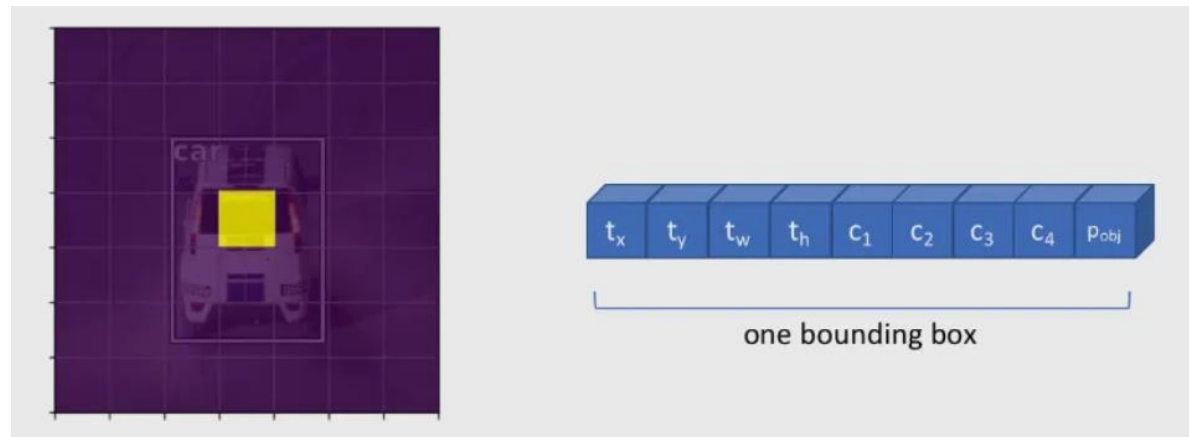
Mechanism:

1. Resize image: feed once through CNN.
2. Divide image into an $S \times S$ grid (e.g., 7×7).
3. Each grid cell predicts:
 1. B boxes (x, y, w, h , confidence)
 2. C class probabilities.



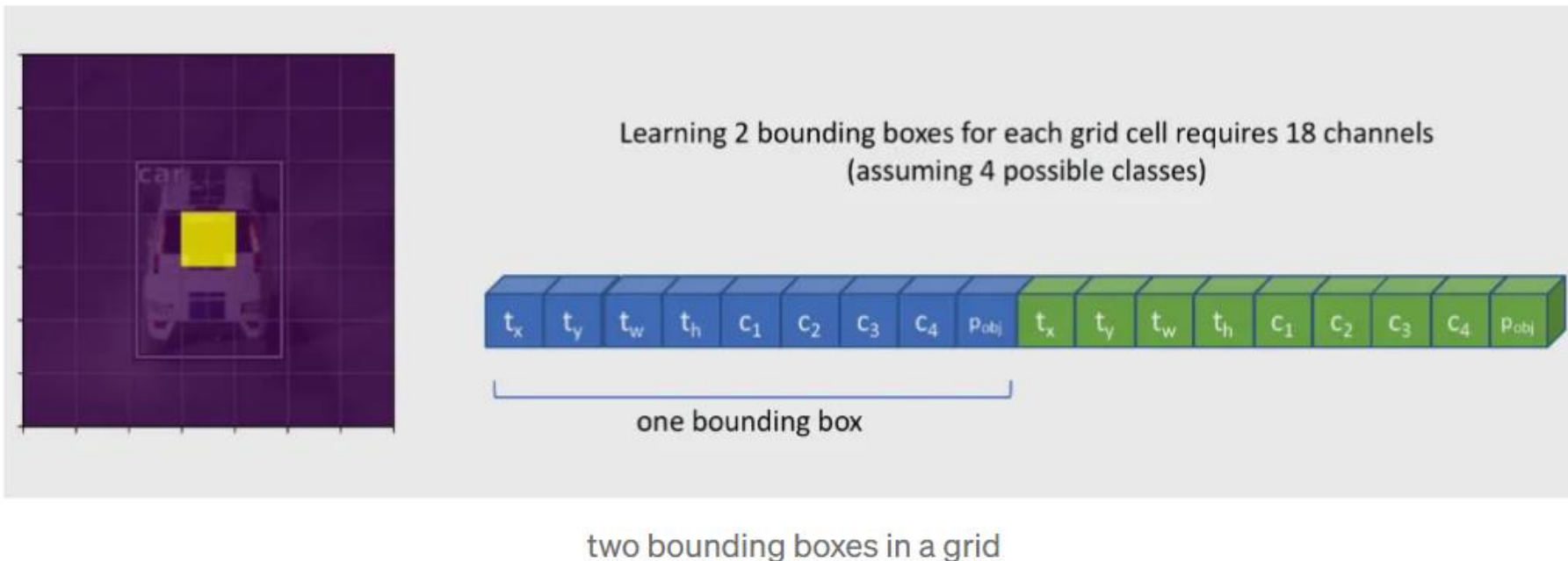
Information predicted for each grid cell in the feature map

- The prediction for each grid cell combines several elements:
 1. The likelihood that the cell contains an object, known as the confidence score or objectness.
 2. The class probabilities, indicating which class the object belongs to.
 3. The coordinates and dimensions of the bounding box that localizes the object.



B bounding box for each cell

- For each grid cell, if we predict B bounding boxes, each bounding box prediction includes five components



Note: the two bounding boxes ($B=2$) predicted by each cell in YOLO do not necessarily correspond to two different objects. They are two different guesses or hypotheses for what the object *might* look like, usually differing in shape (aspect ratio) or size.

How YOLO Works

Input: Resize image to 448×448

- **Backbone CNN:** Extract features (Conv layers)
- **Prediction Layer:** Fully connected layer outputs (x, y, w, h, conf, classes) for each cell.
- **Filtering:**
 - Keep boxes with high confidence (e.g., > 0.5)
 - Apply Non-Maximum Suppression (NMS) to remove overlapping boxes

Output:

A set of labeled boxes: *final detected objects*.

YOLO Loss Function

- YOLO learns to optimize localization and classification and confidence jointly.

$$L = L_{coord} + L_{conf} + L_{class}$$

$$L_{coord} = \lambda_{coord} \sum (x - \hat{x})^2 + (y - \hat{y})^2 + (\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2$$

$$L_{conf} = \sum (C - \hat{C})^2, \quad L_{class} = \sum (p(c) - \hat{p}(c))^2$$

YOLO Loss Function

- YOLO divides the image into an $S \times S$ grid (e.g., $7 \times 7 = 49$ cells).
- For each cell:
 - It predicts **B** bounding boxes
each box has $(x, y, w, h, confidence)$
 - It also predicts **C** class probabilities
- Then, YOLO's total loss is **the sum over all cells**:

$$L = \sum_{i=1}^{S^2} L_{cell_i}$$